Jakarta EE Hackathon Challenge

Title: The "Know-Me" Engine

Short Description (3-5 lines):

The Mission: Build a portable, enterprise-grade decision-support system using the **Jakarta EE 11** platform. Your application must ingest data from a Large Language Model (LLM) and refine it through a structured, context-aware filter to provide hallucination-free guidance that bridges the gap between **vast public knowledge** and **personal context.**

The Goal: Eliminate the "Average Answer." We want a system that doesn't just know *about* the world. It knows why that information matters to *you.*

Potential Use Cases:

- **The Curated Library:** Beyond "good books," find the next read based on my specific philosophical interests and past favorites.
- **The Life Navigator:** Naming a child, picking a career path, or choosing a travel destination based on specific cultural, budgetary, or emotional constraints.
- **The Taste Maker:** A music or movie engine that explains *why* a suggestion fits your unique "vibe" profile.

The Tech Stack Requirements

- **Runtime:** Must run on a Jakarta EE compatible runtime (e.g., WildFly, Payara, GlassFish, or Open Liberty).
- **AI Integration:** Use **LangChain4j-CDI** or **OmniHai** for interacting with LLMs.
- **Portability First:** Prioritize standardized Jakarta APIs (CDI, Jakarta REST, JSON-B/P, Jakarta Persistence) over vendor-specific libraries.

Hints

1. Leverage CDI for "Pluggable" AI

Don't hardcode your LLM providers. Use **CDI Alternatives** or **Specializes** to swap between a local model (for testing) and a cloud-based LLM (for the final demo).

- **Hint:** Define an `@AiService` interface and let **LangChain4j-CDI** handle the implementation. This keeps your business logic clean and independent of the specific AI provider.

2. Standardize Your "Structured Information"

The prompt asks for "structured information." Don't just return a massive String. Use **Jakarta JSON Binding (JSON-B)** to map LLM responses directly into Java Records or POJOs.

- **Hint:** If the AI suggests names for a baby, have it return a JSON array with fields for `name`, `origin`, and `meaning`. This allows your Jakarta EE app to sort, filter, or store that data in a database using **Jakarta Persistence** or **Jakarta Data**.

3. Use "System Messages" to Guardrail the AI

To avoid "nonsense answers," use a **System Prompt** to define the AI's persona and constraints.

- **Hint:** "You are a professional genealogist. Only provide baby names that exist in historical records. If you don't know the origin, say 'Unknown'—do not make it up."

4. Context is King: The "Know-Me" Filter

Since the goal is an AI that "knows you," create a **Jakarta Persistence** entity for the "User Profile."

- **Hint:** When the user asks a question, fetch their profile from the DB first, then inject that context into the prompt.

You'll Get

- Installation instructions for the environment
  - Java Development Kit 25 (adoptium.net/marketplace)
  - Maven 3.9.x (https://maven.apache.org/download.cgi)

- A skeleton project to get you started (not required to use if you prefer to use the Starter or build from scratch yourselves)
  - https://github.com/ivargrimstad/duke-knows-me

- API Key for OpenAI